

Экспериментальный стенд: каскадная архитектура интеграции разрешения кореференции в построение семантического графа

Схема пайплайна первой очереди экспериментального стенда: от золотого корпуса с разметкой кореференции до сравнительных метрик качества графа и кореференции.

Область охвата документа:

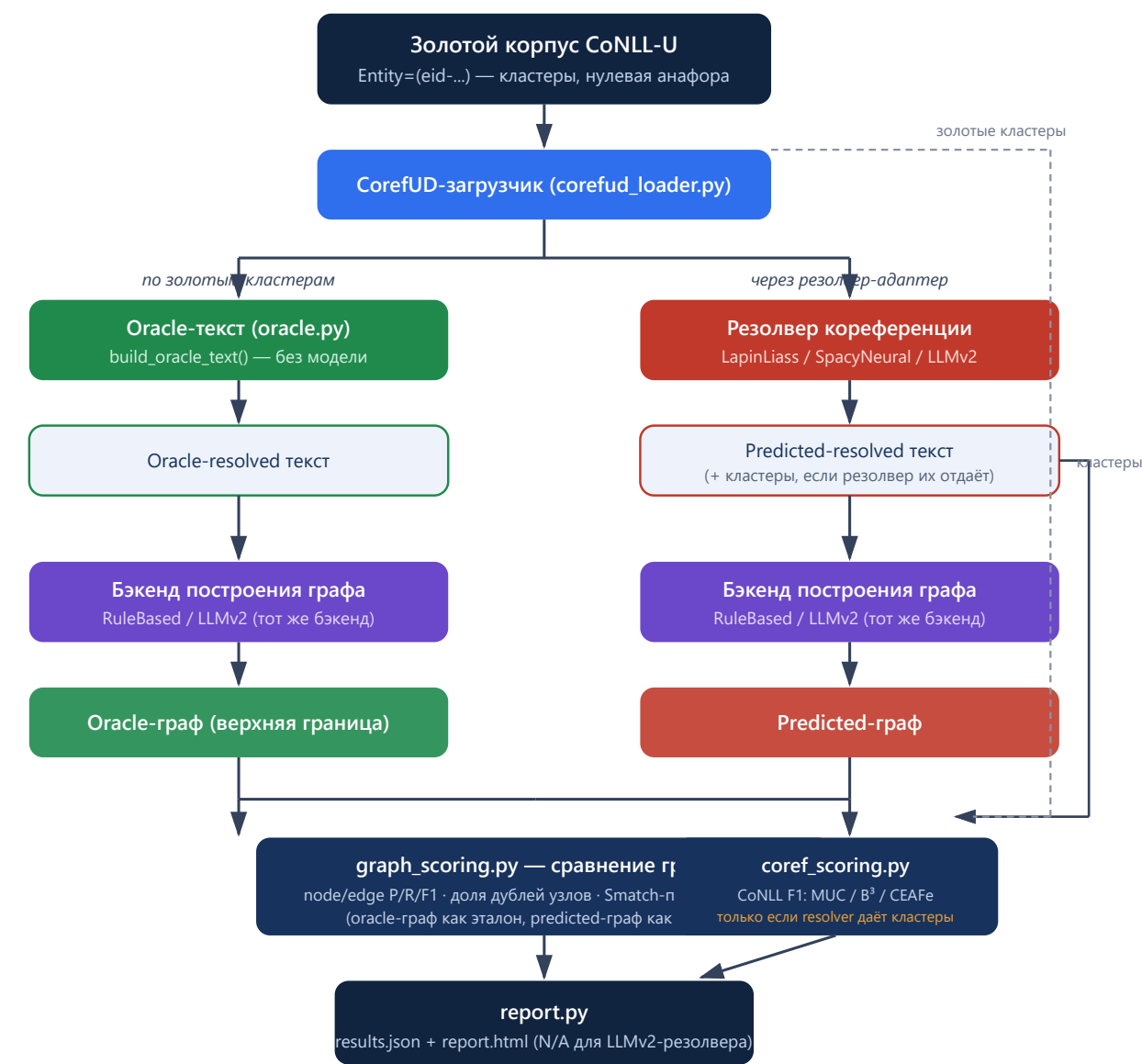
Каскадная архитектура (задачи 2–3 плана диссертации) — унифицированный стенд и эталонный каскадный конвейер, построенный на переиспользуемом коде репозитория `text-corpora-processing`. Совместная (joint) архитектура, мультиязычное сравнение, статистическая значимость и нисходящая QA-задача — предмет последующих этапов (см. раздел «Границы охвата и дальнейшие шаги»).

1. Назначение экспериментального стенда

Экспериментальный стенд автоматизирует один прогон исследовательского эксперимента: на входе — корпус с золотой разметкой кореференции в формате CoNLL-U (нотация CorefUD, атрибут `Entity=`), на выходе — количественное сравнение качества разрешения кореференции и качества построенного из текста семантического графа для одной или нескольких пар «резолвер кореференции × бэкенд построения графа».

Ключевая идея — **oracle-абляция**: для каждого документа строится не только граф по тексту, обработанному реальным резолвером («предсказанный» граф), но и граф по тексту, в котором все кореферентные упоминания заменены на форму первого упоминания кластера с использованием *золотой* разметки («оракульный» граф, верхняя граница качества). Разрыв между оракульным и предсказанным графами — это «бюджет ошибок», вносимых несовершенством разрешения кореференции в построение графа, — центральная величина, вокруг которой строится вся диссертационная гипотеза H1.

2. Архитектурная схема пайплайна



Условные обозначения

- Загрузка и парсинг корпуса (переиспользование не требуется)
- Oracle-ветка — без модели, строится напрямую из золотой разметки
- Predicted-ветка — реальный резолвер, переиспользованный из text-corporuses-processing
- Бэкенд построения графа — общий для обеих веток одного прогона
- Модули оценки (eval/) — coval и smatch как настоящие зависимости
- Данные золотой разметки, используемые напрямую при оценке

Все обращения к коду text-corporuses-processing — через sys.path (add_text_corpuses_processing_to_path), без изменения кода репозитория-источника; ленивые импорты внутри resolve()/build().

Рис. 1. Схема данных каскадного экспериментального стенда: от золотого корпуса к сравнительным метрикам.

3. Компоненты пайплайна

Модуль	Назначение	Источник
pipeline/corpus/corefud_loader.py	Разбор CoNLL-U с нотацией <code>Entity=</code> (открывающие/закрывающие скобки, составные фрагменты на одном токене, пустые узлы нулевой анафоры) в объекты <code>CorefDocument</code> с золотыми кластерами.	Написан для стенда
pipeline/corpus/oracle.py	Построение oracle-текста: замена всех нецентральных упоминаний кластера на форму первого (по порядку в документе) упоминания.	Написан для стенда
pipeline/resolvers/*_adapter.py	Адаптеры трёх резолверов кореференции (LapinLiass, SpacyNeural, LLMv2) к единому интерфейсу <code>resolve(doc) → ResolverOutput</code> , с проекцией упоминаний на золотую токенизацию.	Переиспользование text-corpus-es-processing
pipeline/graph/*_adapter.py	Адаптеры двух бэкендов построения графа (RuleBased, LLMv2) к единому интерфейсу <code>build(text) → dict</code> .	Переиспользование text-corpus-es-processing
pipeline/eval/coref_scoring.py	CoNLL F1 (MUC, B ³ , CEAF _e) через библиотеку <code>coval</code> — эталонный академический скорер кореференции.	Написан для стенда, реальная зависимость coval
pipeline/eval/graph_scoring.py	Precision/Recall/F1 по узлам и рёбрам (с сохранением направления рёбер), доля дублирующихся узлов, Smatch-подобный структурный балл через <code>penman_convert.py</code> и библиотеку <code>smatch</code> .	Написан для стенда, реальная зависимость smatch
pipeline/report.py, run_experiment.py	Оркестрация всего прогона и генерация <code>results.json</code> + HTML-отчёта; CLI-точка входа.	Написан для стенда

4. Метрики оценки

Кореференция (при наличии кластеров у резолвера)

- **MUC** — по связям между упоминаниями
- **B³** — по пересечению кластеров на уровне упоминаний
- **CEAF_e** — по оптимальному сопоставлению кластеров
- **CoNLL F1** — среднее по трём метрикам выше

Золотые синглтон-кластеры исключаются перед подсчётом (стандарт CoNLL-2012), иначе оценка резолвера, структурно не выдающего синглтоны, занижается.

Граф (oracle vs. predicted)

- **Node P/R/F1** — по множеству меток узлов
- **Edge P/R/F1** — по тройкам (узел₁, узел₂, метка), с сохранением направления
- **Доля дублей узлов** — доля узлов с повторяющейся меткой внутри одного графа
- **Smatch-подобный балл** — оптимальное сопоставление узлов/рёбер (AMR-алгоритм)

Разрыв между значением метрики на oracle-графе и на predicted-графе — это оценка «бюджета ошибок», вносимого несовершенством этапа разрешения кореференции в каскадной архитектуре (гипотеза H1 диссертационного плана).

5. Границы охвата текущей версии и дальнейшие шаги

Реализовано в стенде v1	Вне рамок (последующие этапы)
• Унифицированный мультязычный загрузчик CorefUD задача 2 • Каскадная архитектура: три резолвера × два бэкэнда графа задача 3 • Oracle-абляция как встроенный инструмент сравнения • CoNLL F1 и Smatch-подобная оценка на реальных библиотеках	• Совместная (joint) архитектура задача 4 • Мультязычное пакетное сравнение и сводные таблицы по языкам • Статистическая значимость (bootstrap, поправка Холма) • Кривые обучения для гипотезы H3, анализ ошибок по типам анафоры • Нисходящая вопросно-ответная задача по графу

Резолвер LLMv2 переписывает текст, но не отдаёт явные спаны упоминаний — для него считается только графовая (oracle-абляция) метрика; кореференционный CoNLL F1 в этом случае помечается как «N/A» в отчёте, что явное, а не скрытое ограничение архитектуры.

Документ сгенерирован автоматически по результатам реализации экспериментального стенда (15 задач плана + финальный сквозной ревью, 79/79 тестов). Каталог реализации: [coreference/pipeline/](#). Спецификация: [docs/superpowers/specs/](#). План реализации: [docs/superpowers/plans/](#).